

Neural Networks, Deep Learning and Applications

Generative AI

Moritz Handrejk, Tim Rothe, Helmi Ghliiss, Malak Mrini
Team Lasagna

TU Dresden X FSD
Summer Semester 2025

Project Overview

- Objective: Classify braking system conditions using sensor data
- Approach: Deep learning models + Synthetic data generation
- Data: 6-dimensional sensor input (128 time steps), 12 braking condition classes

Outline

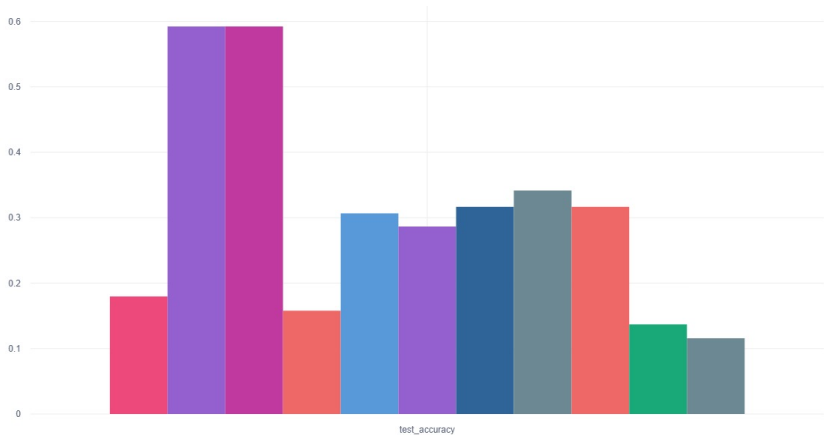
- 1 Classification models
 - LSTM
 - FCN
 - CNN
- 2 Generative models
 - Evaluation
 - GAN
 - RNN Generator
- 3 Conclusions and interpretation

Classification models

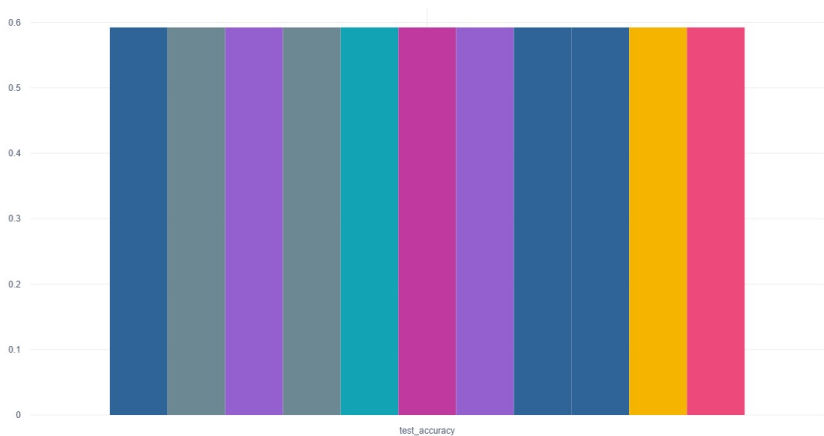
LSTM Model Architecture

- LSTM layers $\rightarrow$ FC layer $\rightarrow$ 12 output values

Accuracies



Accuracies



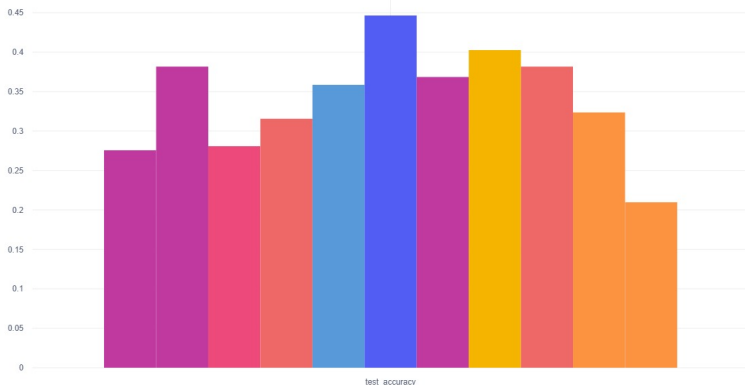
Data distributions

label	
0	1798
6	294
3	286
1	273
7	258
8	253
4	236
2	227
5	225
9	94
10	89
11	20

label	
0	593
2	105
1	104
4	103
5	96

$$\mathcal{L}_{\text{WCE}} = - \sum_{i=1}^C w_i \cdot y_i \log(\hat{y}_i)$$

Final accuracies



- Final model: **hidden dimension: 128; layers: 2; batch size: 32; learning rate: 0.0001; weight decay: 0.0001**
- Accuracy: 34.69%

FCN : Model Architecture

- **Flatten Layer** Input tensor from shape $(128, 6)$ to a flat vector of size $768 = 6 \times 128$.
- **Linear Layer** Fully connected layer: $768 \rightarrow 128$
- **ReLU Activation** Applies the ReLU function:
- **Linear Layer** Fully connected layer: $128 \rightarrow 64$
- **ReLU Activation** Again, ReLU function.
- **Output Linear Layer** Fully connected layer: $64 \rightarrow 12$

Parameter

The following hyperparameter combinations were used:

- **Optimizers:** SGD, Adam, RMSprop, Adagrad
- **Learning rates:** 0.001, 0.005, 0.01, 0.015
- **Epochs:** 7
- **Batch sizes:** 32, 64

Training accuracy

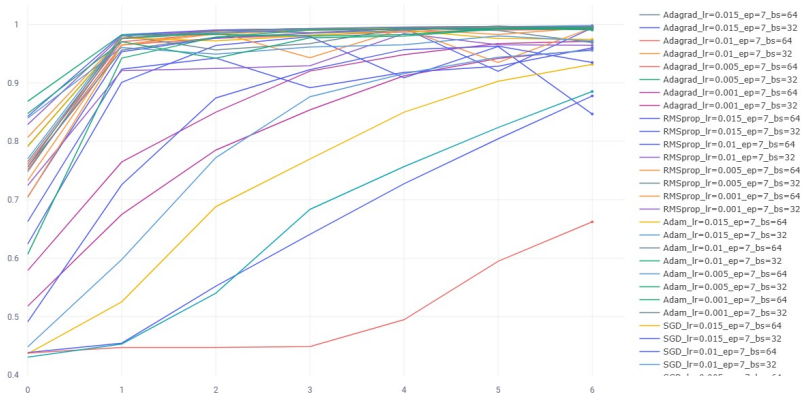


Figure: Training accuracy pro Epoch

Random split

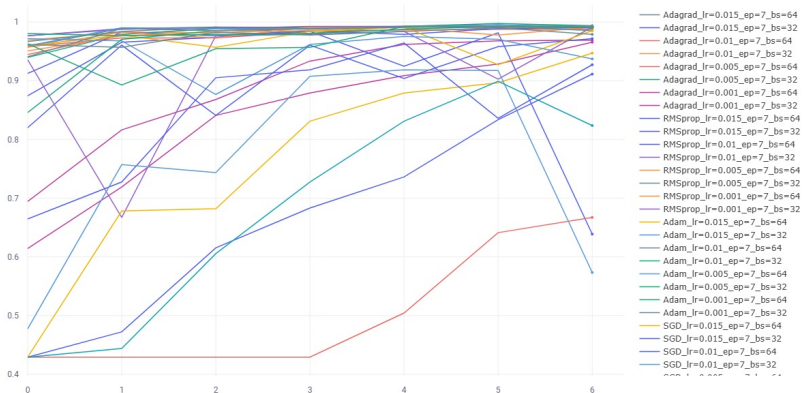


Figure: Random split accuracy pro Epoch

Testing accuracy

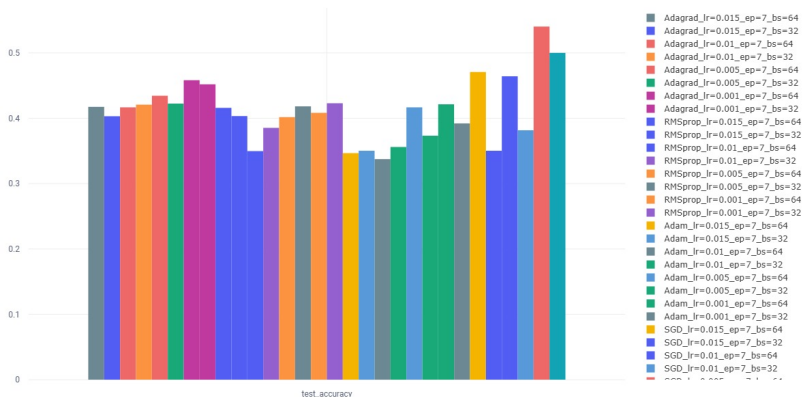


Figure: Testing accuracy for each model in average

Accuracy reaches a maximum of 0.54 for the SGD $lr=0.005$ and $bs=64$.

2D CNN : Model Architecture

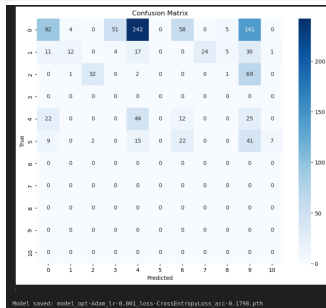
- **Conv1**: 32 filters (3x3), ReLU, MaxPool (2x2)
- **Conv2**: 64 filters (3x3), ReLU, MaxPool (2x2)
- Flatten (1D vector of size 2048) $\rightarrow$ Dense(128) $\rightarrow$ Dense(12 classes)
- **Output** : 12 predictions

Initial Results: Overfitting

Test Accuracy: 0.1798

Classification Report:

	precision	recall	f1-score	support
0	0.6866	0.1551	0.2531	593
1	0.7059	0.1154	0.1983	104
2	0.9412	0.3048	0.4604	105
3	0.0000	0.0000	0.0000	0
4	0.1375	0.4272	0.2080	103
5	0.0000	0.0000	0.0000	96
6	0.0000	0.0000	0.0000	0
7	0.0000	0.0000	0.0000	0
8	0.0000	0.0000	0.0000	0
9	0.0000	0.0000	0.0000	0
10	0.0000	0.0000	0.0000	0
accuracy			0.1798	1001
macro avg	0.2246	0.0911	0.1018	1001
weighted avg	0.5929	0.1798	0.2402	1001

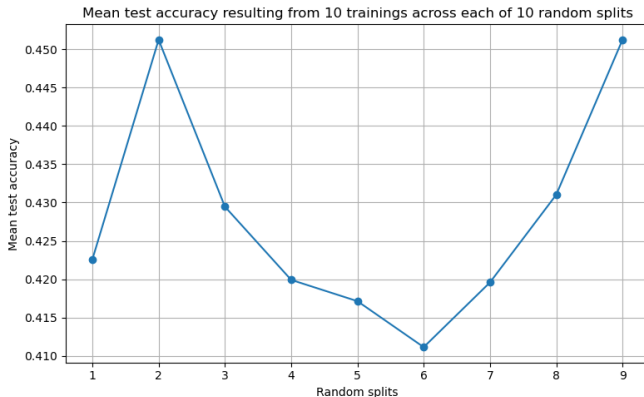


- **Training accuracy:** 99% in 5 epochs, rapid
- **Test accuracy:** only 17.98%, lack of generalization
- Lack of reliability, low recall → many missed true instances
- **Bias** towards dominant classes
- **Overfitting**, class imbalance

Hyperparameter Tuning

- Used *dropout*, *class weights*, *early stopping*, *80/20 random splits*
- **Targeted hyperparameters** : optimizer, learning rate, batch size, and dropout rate
- Grid search, around **100** hyperparameter configurations tested
- **Best hyperparameter combination** : *Adam* optimizer, $lr = 0.001$, batch size = 64, and dropout rate = 0.3, training time 25 sec $\rightarrow$ **50.25%** test accuracy

Evaluation of reliability of the results



- 10 random splits, 10 runs each → mean test acc around **42%**
- Performance variability → data distribution, randomness of splits
- Some harder-to-learn patterns

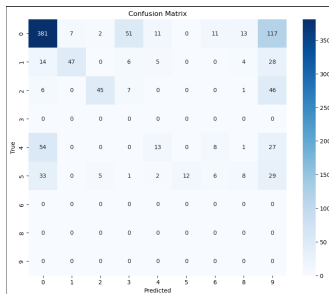
Best accuracy achieved

Training completed in 23.53 seconds

Test Accuracy: 0.4975

Classification Report:

	precision	recall	f1-score	support
0	0.7807	0.6425	0.7049	593
1	0.8704	0.4519	0.5949	104
2	0.8654	0.4286	0.5732	105
3	0.0000	0.0000	0.0000	0
4	0.4194	0.1262	0.1940	103
5	1.0000	0.1250	0.2222	96
6	0.0000	0.0000	0.0000	0
8	0.0000	0.0000	0.0000	0
9	0.0000	0.0000	0.0000	0
accuracy			0.4975	1001
macro avg	0.4373	0.1971	0.2544	1001
weighted avg	0.7828	0.4975	0.5808	1001



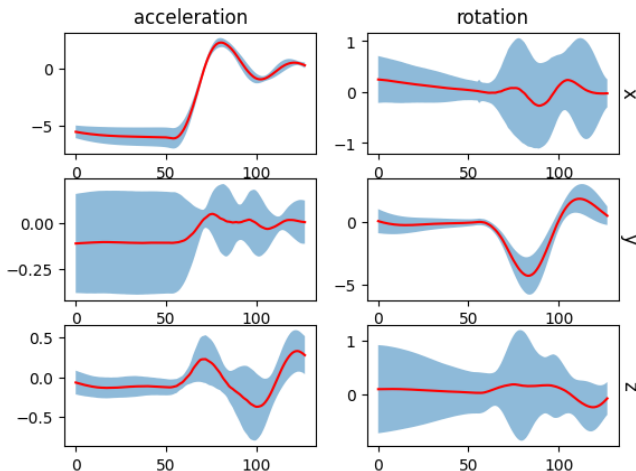
- **Best test accuracy : 49.75%**
- Strong on common classes (0,1,2)
- Misclassification still common for rare classes
- Training dataset $\neq$ test dataset

Comparison and conclusion

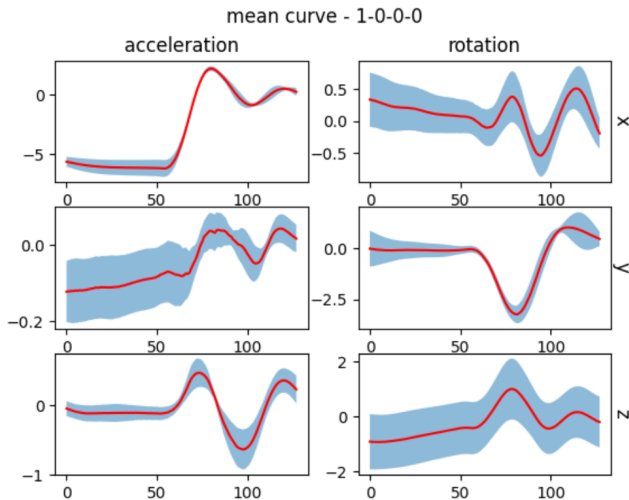
- **LSTM** : at first affected by class imbalance, test acc avg **34.69%**
- **FCN** : best average test performance (54%)
- **2D CNN** : peaked at around 50%, stable around **42%**
- **Chosen model:** FCN and 2D CNN
- **Next step:** Boost performance with synthetic data from generative models

Evaluation

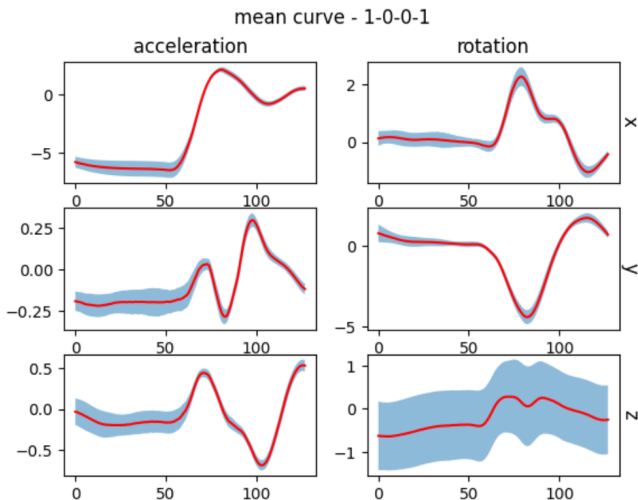
The mean curve



Mean curve per classification



Mean curve per classification



Our metric

Metric: Least squares distance to class-specific mean curve

Limitations:

- small sample sizes
- not adjusted for large deviation
- does not capture temporal correlations

Generative Adversarial Networks

Architecture – Generator

$\text{ConvTranspose1d} \rightarrow \text{BatchNorm1d} \rightarrow \text{ReLU} \times 4$

- latent space 100
- embedding space 16
- no normalization on the last layer

Architecture – Discriminator

$\text{Conv1d} \rightarrow \text{BatchNorm1d} \rightarrow \text{LeakyReLU} \times 3$

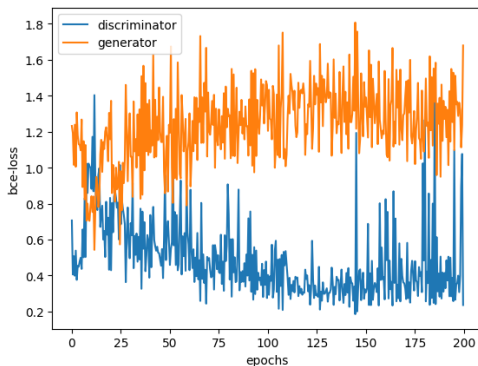
- embedding space 6
- LeakyReLU with negative slope 0.2
- last layer sigmoid activation added to distinguish between labels

Architecture – Other things were tried

LSTM-based approach was tried but abandoned due to:

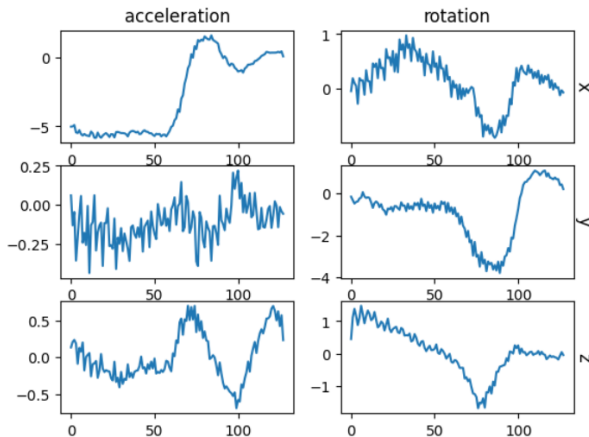
- low gradients for early time steps (due to architecture)
- low generator quality due to small gradients in discriminator training
- generator susceptible to mode collapse (all outputs look very similar)
- specifics can be found and tried in our repository

Training



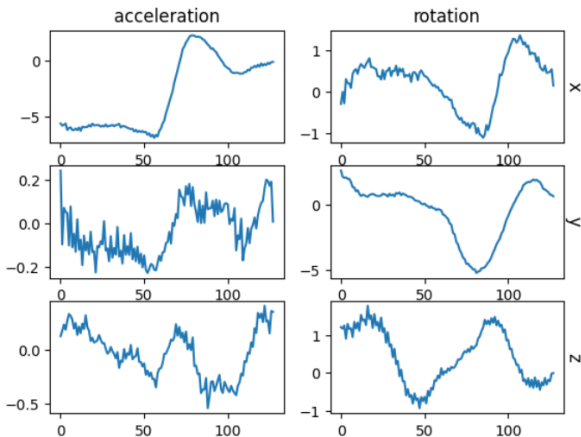
- learning rate 0.0001
- two discriminator optimization steps for one generator step
- 200 epochs

Results – After 50 epochs



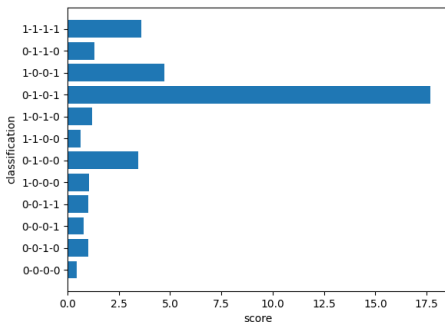
EPOCH 51 @ θ : discriminator 0.5563 generator 1.1496

Results – After 200 epochs



EPOCH 201 @ θ : discriminator 1.4102 generator 0.9714

Results

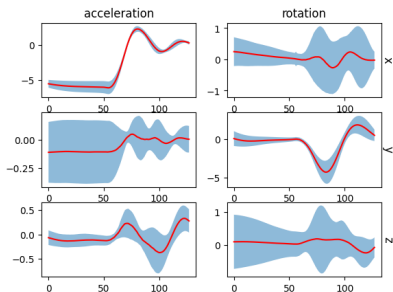


- performs particularly well on certain classes
- on other particularly bad (mode collapse?)
- more training is necessary
- still discontinuities – could be solved by post-processing

RNN Generators

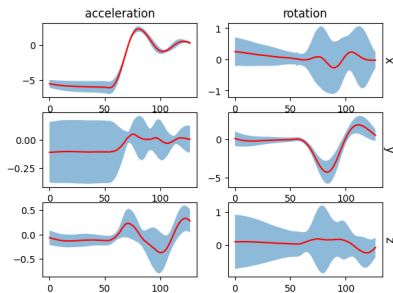
RNN Generator

- LSTM-based sequence generation
- temporal dependency

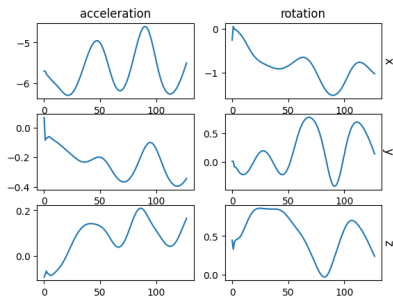


Simple Implementation

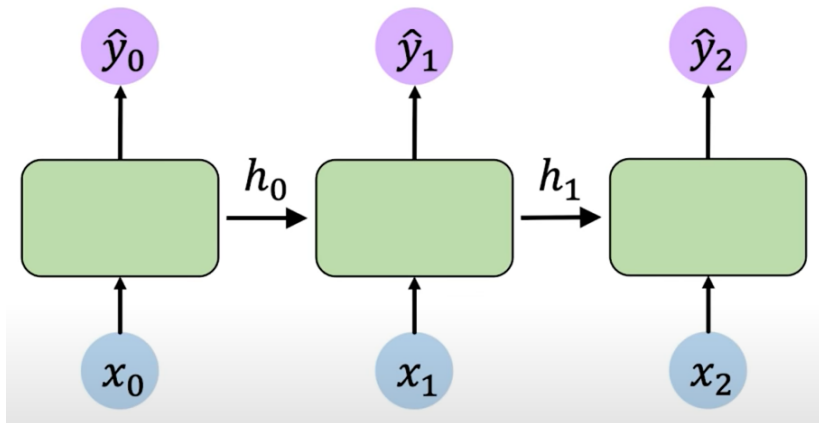
Real Data



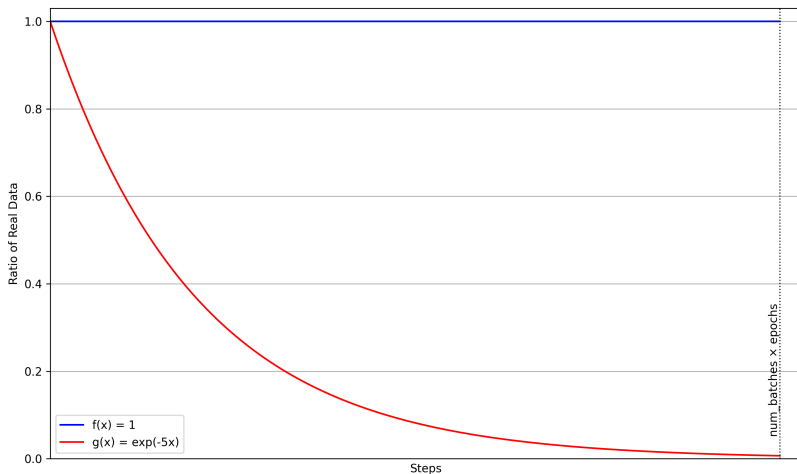
Generated Data



Teacher Forcing

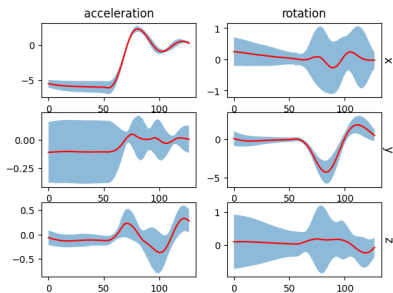


Teacher Forcing Ratio

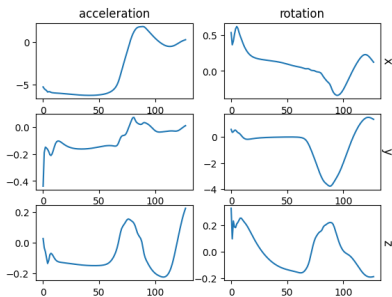


Teacher Forcing

Real Data



Generated Data



Summary

- We explored deep learning models for brake condition classification
- We generated synthetic data via GANs and RNNs
- We added the generated data and tried to improve the testing accuracy

Conclusions

- The random split evaluation was always high for all models.
- The FCN and the 2D CNN were the best-performing architectures for a good choice of parameters
- The LSTM struggled with generalization and class imbalance
- The GAN and the RNN gave good synthetic data
- Adding the synthetic data didn't enhance the classification accuracy The more synthetic data we added the worse the accuracy got. We had accuracy around 0.3

Interpretation

- The high accuracy for the randomsplit evaluation is due to a data leakage (since the random split keeps the data distribution)
- The training data and the test data look different which makes generalization harder
- There is bias in the data
- The synthetic data was generated to be similar to the training data and didn't cover the different sensor data in the testing dataset

Questions

Thank you for your attention!

We welcome your questions.

Exact Architecture – Generator

```

=====
Layer (type:depth-idx)      Output Shape      Param #
=====
CNNGenerator
├─Embedding: 1-1            [32, 128, 6]      --
├─Sequential: 1-2           [32, 16]           192
├─├─Linear: 2-1              [32, 2048]         239,616
├─├─ReLU: 2-2                [32, 2048]         --
├─Sequential: 1-3           [32, 6, 128]       --
├─├─ConvTranspose1d: 2-3     [32, 128, 16]      131,200
├─├─BatchNorm1d: 2-4         [32, 128, 16]      256
├─├─ReLU: 2-5                [32, 128, 16]      --
├─├─ConvTranspose1d: 2-6     [32, 64, 32]       32,832
├─├─BatchNorm1d: 2-7         [32, 64, 32]       128
├─├─ReLU: 2-8                [32, 64, 32]       --
├─├─ConvTranspose1d: 2-9     [32, 32, 64]       8,224
├─├─BatchNorm1d: 2-10        [32, 32, 64]       64
├─├─ReLU: 2-11              [32, 32, 64]       --
├─├─ConvTranspose1d: 2-12    [32, 6, 128]       774
├─├─Identity: 2-13           [32, 6, 128]       --
=====
Total params: 413,286
Trainable params: 413,286
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 128.50
=====
Input size (MB): 0.01
Forward/backward pass size (MB): 3.87
Params size (MB): 1.65
Estimated Total Size (MB): 5.54
=====

```

Exact Architecture – Discriminator

```

=====
Layer (type:depth-idx)                   Output Shape          Param #
=====
CNNDiscriminator                        [32]                  --
├─Embedding: 1-1                        [32, 6]               72
├─Sequential: 1-2                       [32, 1, 1]            --
│   ├──Conv1d: 2-1                      [32, 32, 64]          1,568
│   │   ├──LeakyReLU: 2-2               [32, 32, 64]          --
│   │   ├──Conv1d: 2-3                  [32, 64, 32]          8,256
│   │   │   ├──BatchNorm1d: 2-4         [32, 64, 32]          128
│   │   │   ├──LeakyReLU: 2-5          [32, 64, 32]          --
│   │   │   ├──Conv1d: 2-6             [32, 128, 16]         32,896
│   │   │   ├──BatchNorm1d: 2-7        [32, 128, 16]         256
│   │   │   ├──LeakyReLU: 2-8          [32, 128, 16]         --
│   │   │   ├──Conv1d: 2-9             [32, 256, 8]          131,328
│   │   │   ├──BatchNorm1d: 2-10       [32, 256, 8]          512
│   │   │   ├──LeakyReLU: 2-11        [32, 256, 8]          --
│   │   │   └──Conv1d: 2-12            [32, 1, 1]            2,049
└─Total params: 177,065
└─Trainable params: 177,065
└─Non-trainable params: 0
└─Total mult-adds (Units.MEGABYTES): 62.22
=====
Input size (MB): 0.10
Forward/backward pass size (MB): 3.67
Params size (MB): 0.71
Estimated Total Size (MB): 4.48
=====

```